



Technical Documentation

Strict QEMU Launcher

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

**BMW
GROUP**



ROLLS-ROYCE
MOTOR CARS LTD

[BMW Car IT, Ulm](#)

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

The `qemu_tests.sh` script serves as the Strict QEMU Launcher dedicated to the automated execution environment. It is designed ONLY for automated testing by the `run_verity_tests.sh` test harness and should NOT be executed manually for development or debugging purposes. Its primary function is to launch QEMU with specific, non-interactive parameters to ensure deterministic, reproducible results for the security test suite.

Exclusive Automated Use

This script enforces a strict separation from manual workflows. Developers should utilize `src/build/launch_qemu.sh` for interactive debugging or `src/bootloaders/qemu_main.sh` for minimal manual launches.

The script's automated focus is evident in its reliance on positional arguments and its strict error handling:

```
30  set -u # treat unset vars as an error
31
32  # Get script directory for relative paths
33  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
34
35  # Use the first argument as the disk image (no fallback so the original rootfs.img does not get overwritten)
36  if [[ -z "$1" ]]; then
37      echo "ERROR: No disk image specified" >&2
38      echo "Usage: $0 <disk_image> [serial_log_file]" >&2
39      exit 1
40  fi
```

- **set -u:** Ensures any use of an uninitialized variable is treated as a fatal error, which is critical for deterministic automation.
- **Argument Requirement:** It requires the path to the disk image (\$1) and optionally the serial log file path (\$2) to be passed by the calling automation script (`run_verity_tests.sh`).

1. Execution Environment Configuration

The script performs several critical setup steps before launching QEMU to guarantee a reproducible test environment.

A. Input and Path Validation

The script rigorously validates all inputs and converts paths to absolute format, preventing issues related to the Current Working Directory (CWD).

```
47  # Resolve absolute path for disk image
48  if ! IMG_ABS=$(realpath "$IMG_REL"); then
49      echo "ERROR: realpath($IMG_REL) failed" >&2
50      exit 1
51  fi
52
53  # Check if disk image exists BEFORE doing anything else
54  if [[ ! -f "$IMG_ABS" ]]; then
55      echo "ERROR: Disk image not found: $IMG_ABS" >&2
56      exit 1
57  fi
```

This strict path resolution and file existence check ensures that the QEMU command uses exactly the test image intended by `run_verity_tests.sh`.

B. Kernel Discovery

To maintain flexibility across different build systems, the script searches for the required Linux kernel image (bzImage) in multiple standard locations relative to the project structure.

The first valid kernel file found is used, ensuring the correct kernel version is always loaded for the integrity tests.

2. QEMU Launch and Command Line

The QEMU invocation is tailored specifically for the dm-verity testing module.

A. Non-Interactive Mode

The core QEMU parameters are set to eliminate any interactive elements, which is essential for execution under the automated timeout utility:

- `-nographic`: Suppresses the graphical display.
- `-nodefaults`: Avoids loading default peripherals.

B. Command Line Argument Injection

The script constructs the kernel command line (`APPEND_CMDLINE`) to instruct the kernel to use the custom `dm-verity-autoboot` module for the root filesystem:

```
87 APPEND_CMDLINE="console=ttyS0,115200 \  
88 loglevel=7 \  
89 dm_verity_autoboot.autoboot_device=/dev/vda \  
90 root=/dev/dm-0 rootfstype=ext4 rootwait rootdelay=10"
```

- `dm_verity_autoboot.autoboot_device=/dev/vda`: This parameter is the critical instruction that triggers the execution of the `dm_verity_autoboot.c` module's main worker function after a 5-second delay.
- `root=/dev/dm-0`: Instructs the kernel to mount the root filesystem from the device mapper target, which will only exist if the `dm_verity_autoboot` module successfully verified and set up the device.

C. Serial Output Management

The script dynamically configures the serial output based on whether the test harness provided a log file path (\$2):

```
98 # Configure serial output based on whether we're in automated mode  
99 if [[ -n "$SERIAL_LOG" ]]; then  
100     echo "Serial output: ${SERIAL_LOG}"  
101     SERIAL_OPT="-serial file:${SERIAL_LOG}"  
102 else  
103     echo "Serial output: stdio"  
104     SERIAL_OPT="-serial mon:stdio"  
105 fi
```

When called by `run_verity_tests.sh`, **SERIAL_OPT** directs all boot output into a file, which is then parsed by `run_verity_tests.sh` to determine the security outcome (KERNEL_PANIC, REJECTED, or BOOT_SUCCESS). When run manually (without \$2), it defaults to `mon:stdio`, printing output to the console, although this is strongly discouraged.

Execution Summary and Status

The script concludes by echoing the final exit status of the QEMU process:

```
124 STATUS=$?  
125 echo ""  
126 echo "=====  
127 echo "QEMU exited with status: ${STATUS}"  
128 echo "=====  
129 exit $STATUS
```

This exit status is captured by the calling `run_verity_tests.sh` script to confirm whether a time-out (124) or an early crash occurred, which forms the basis of the final pass/fail test evaluation.

End of Document